

Foundations: Function Approximation, Classification, and the Multi-Layer Perceptron

This library builds a neural network in pure Python, no NumPy, no matrix libraries. It exists to make one progression concrete: the network as a parameterized function approximator, the single linear unit and its expressive limits, and the multi-layer perceptron that escapes them.

Throughout this post (and the posts that follow it) we are doing **supervised learning**: we have labeled (x, y) pairs and learn f such that $f(x) \approx y$. The labels can come from humans (classical supervised) or from the data's own structure (self-supervised, as in language modeling). Both are mathematically the same. What differs across sections is the *interpretation* of the output: the choice of output activation (link function) and matching loss.

1. What is a neural network?

A neural network is a parameterized function $f_\theta : \mathbb{R}^n \rightarrow \mathbb{R}^m$. It takes a real-valued input vector and produces a real-valued output vector. That is the whole structural commitment. What the output *means* is a separate decision, made by the loss.

The network produces raw values; the loss interprets those values as a prediction by composing them with an **output activation** (also called a **link function** in the language of generalized linear models). The choice of link function and matching loss together specify what kind of thing the network is predicting.

The simplest case is **regression**: the raw output *is* the prediction. The link function is the identity, and the loss is **mean squared error**:

$$L = \frac{1}{2} \sum_i (\hat{y}_i - y_i)^2.$$

The gradient with respect to the raw output is just the residual:

$$\frac{\partial L}{\partial \hat{y}_i} = \hat{y}_i - y_i.$$

This pattern, $\hat{y} - y$, recurs throughout the post. It is the gradient of *every* canonical loss with respect to *its* raw output, for reasons that turn out to be a deep theorem about exponential families and canonical links, not an accident.

A concrete demonstration: fit $y = \sin(x) + \varepsilon$ on $x \in [-\pi, \pi]$ with a small MLP, identity output, and MSE loss. After a few thousand epochs of mini-batch SGD, the network learns the curve.

```
net = Network([Linear(1, 32), Tanh(),
               Linear(32, 1)], loss=MSELoss())
```

This is `examples/regression.py`. The architecture is the same MLP we build up to in section 3. Only the output head and loss differ.

With regression in hand as the unspecialized base case, the rest of the post is a tour through two questions. First, what *expressive limits* does the architecture itself impose? That is sections 2 and 3: a single linear unit cannot represent all continuous functions; the multi-layer perceptron

can. Second, what *output-side specializations* let the same architecture solve concrete prediction problems beyond regression? That is sections 4 and 5: classification by sigmoid for binary outcomes, softmax for categorical.

2. The single linear unit and its limits

The smallest non-trivial network is one **Linear** layer mapping inputs to outputs:

$$\mathbf{z} = W\mathbf{x} + \mathbf{b}.$$

This is a **single-layer perceptron** (SLP): an affine map with no non-linearity. Trained with MSE, it learns the least-squares fit. Trained with a sigmoid head and BCE loss (section 4), it is logistic regression. The function class either way is *linear* in the input.

The SLP can fit any linearly separable target. It cannot fit anything else.

The canonical demonstration is the XOR function on $\{0, 1\}^2$ with output the XOR of its inputs:

$$\text{XOR}((0, 0)) = 0, \quad \text{XOR}((0, 1)) = 1, \quad \text{XOR}((1, 0)) = 1, \quad \text{XOR}((1, 1)) = 0.$$

Try to fit this with a single linear unit and a sigmoid head:

```
slp = Network([Linear(2, 1)], loss=SigmoidBCE())
slp.fit(xor_X, xor_Y, epochs=500, lr=0.5, batch_size=4)
```

Every input converges to $p \approx 0.5$. The model literally cannot do better. To see why, write down what fitting all four points would require. With $\hat{y} = \sigma(w_1x_1 + w_2x_2 + b)$ and targets $\{0, 1, 1, 0\}$, the BCE loss is minimized when each $\hat{y}$ matches its target. Sigmoid is monotonic, so the four inequalities reduce to:

$$b < 0, \quad w_1 + b > 0, \quad w_2 + b > 0, \quad w_1 + w_2 + b < 0.$$

Adding the second and third gives $w_1 + w_2 + 2b > 0$, so $w_1 + w_2 + b > -b > 0$. But the fourth says $w_1 + w_2 + b < 0$. Contradiction. No choice of (w_1, w_2, b) fits all four points.

Geometrically: a linear map produces a single decision boundary (a line in 2-D); XOR's positive class $\{(0, 1), (1, 0)\}$ and negative class $\{(0, 0), (1, 1)\}$ are not separable by any line. The four points are arranged at the corners of a unit square, and the two positive corners are diagonal to each other. No line through the square can separate them.

This was Minsky and Papert's argument in 1969 that killed the perceptron research program for the next decade and a half. The resolution is one of the cleanest examples of how architecture and expressivity interact. To escape linearity, add a non-linearity.

This demo is `demo_mlp_xor()` in `examples/demos.py`.

3. The multi-layer perceptron, and what depth and non-linearity buy

The multi-layer perceptron (MLP) stacks **Linear** layers with a non-linear **activation** between them (**Tanh** or **ReLU**). Without the non-linearity, stacked linear maps collapse: two affine maps composed are again affine, and no number of stacked **Linear** layers without non-linearities buys anything over one.

```
mlp = Network([Linear(2, 8), Tanh(),
               Linear(8, 1)], loss=SigmoidBCE())
mlp.fit(xor_X, xor_Y, epochs=4000, lr=0.5, batch_size=4)
```

Now the MLP fits XOR exactly. Mechanically: the hidden layer can construct intermediate features that *are* linearly separable. One natural decomposition for XOR is

$$\text{XOR}(x_1, x_2) = \text{OR}(x_1, x_2) - \text{AND}(x_1, x_2),$$

so a hidden unit detecting OR and another detecting AND give the output layer a linearly separable problem. The training run discovers something equivalent.

The general fact is the **universal approximation theorem** (Cybenko 1989, Hornik 1989): a feedforward network with one hidden layer of sufficient width and any non-polynomial activation can approximate any continuous function on a compact set to arbitrary precision. *Depth is not required for expressivity*; one hidden layer suffices. What deeper networks buy is *efficient* representation: many natural functions that one wide hidden layer can approximate at exponential cost are representable at polynomial cost with a few hidden layers stacked. That is the empirical case for depth; the theoretical case for hidden layers at all is XOR.

The training rule for an MLP is **backpropagation**. The trick is to make the chain rule *local*: each layer implements two methods,

```
forward(x) -> output
backward(g) -> gradient w.r.t. its input    (g = gradient w.r.t. its output)
```

backward also stashes the layer's own parameter gradients. A third method, **parameters()**, hands those weights and their gradients to the optimizer and the gradient checker. A layer never sees the rest of the network. The **Network** runs **forward** through the layers in order, then **backward** through them in reverse, threading the gradient:

```
g = loss.grad(logits, y)          # the familiar p - y from section 4
for layer in reversed(layers):
    g = layer.backward(g)
```

Each layer's **backward** is tiny. For **Linear** ($\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$), the parameter gradients and the gradient passed back are

$$\frac{\partial L}{\partial W_{ij}} = g_i x_j, \quad \frac{\partial L}{\partial b_i} = g_i, \quad \frac{\partial L}{\partial x_j} = \sum_i W_{ij} g_i.$$

For **Tanh**: $\mathbf{g} \odot (1 - \tanh^2(\mathbf{x}))$. For **ReLU**: $\mathbf{g}$ where the input was positive, else 0.

The key realization: the MLP introduced **no new math** beyond what a single linear unit needed. The chain rule applied locally, composed across layers, gives gradients for any depth. Depth is just more composition of the same local steps.

This is the foundation. Everything that follows is the same MLP, with different *interpretations* of its output (classification, sections 4 through 6) or different *architectural priors* on the inputs (CNN, RNN, Transformer, the rest of the series).

4. Logistic regression

A logistic-regression model scores an input $\mathbf{x}$ with a weighted sum, the **logit**:

$$z = \mathbf{w} \cdot \mathbf{x} + b.$$

It squashes that score into a probability with the **sigmoid**:

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}.$$

p is the model's estimate of $P(y = 1 \mid \mathbf{x})$. To train it we need a loss that is small when p is close to the label $y \in \{0, 1\}$. That is **binary cross-entropy**:

$$L = -[y \log p + (1 - y) \log(1 - p)].$$

The gradient of this loss with respect to the logit z is remarkably clean:

$$\frac{\partial L}{\partial z} = p - y.$$

The prediction minus the target. Every weight gradient follows by the chain rule, $\partial L / \partial w_j = (p - y) x_j$, and we descend: $\mathbf{w} \leftarrow \mathbf{w} - \eta \partial L / \partial \mathbf{w}$.

In code, logistic regression is one `Linear` layer with a `SigmoidBCE` loss:

```
net = Network([Linear(n_features, 1)], loss=SigmoidBCE())
```

This is structurally the same SLP from section 2, with a probability interpretation glued on. The SLP's expressive limits still apply: logistic regression can solve any linearly separable binary classification but cannot solve XOR. To solve XOR with sigmoid output, swap the body for the MLP from section 3:

```
net = Network([Linear(2, 8), Tanh(),
               Linear(8, 1)], loss=SigmoidBCE())
```

The output head (sigmoid + BCE) and the architecture (linear vs. MLP) are independent choices, a theme that becomes the load-bearing principle of section 7.

5. Multi-class regression via softmax

With K classes the model produces K logits, one weighted sum per class. We turn the logit vector $\mathbf{z}$ into a probability distribution with the **softmax**:

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}.$$

The matching loss is **categorical cross-entropy**, $L = -\log p_y$, where y is the index of the correct class. Its gradient with respect to the logits is, again,

$$\frac{\partial L}{\partial z_i} = p_i - \mathbb{1}[i = y],$$

that is, $\mathbf{p} - \mathbf{y}$ with $\mathbf{y}$ read as a one-hot vector. The same expression as the binary case. That is no coincidence: binary logistic regression *is* two-class softmax with one logit pinned to zero (softmax depends only on logit differences, so one logit may be fixed freely). The binary sigmoid is the $K = 2$ softmax in disguise.

In code, softmax regression is, again, one **Linear** layer with a categorical head:

```
net = Network([Linear(n_features, K)], loss=SoftmaxCrossEntropy())
```

On the 8x8 UCI optical-recognition digits (3823 training, 1797 test, 10 classes), this one-layer softmax model reaches about 95% test accuracy after 30 epochs of mini-batch SGD. Linear classifiers on this dataset are unexpectedly competitive, which is itself a hint about the data: the 4x4 block-counting that produced these 8x8 features has already done most of the work.

Swap the body for an MLP and the same softmax head:

```
net = Network([Linear(64, 32), Tanh(),
               Linear(32, 10)], loss=SoftmaxCrossEntropy())
```

On the same UCI digits, this 64-32-10 MLP reaches about 96% test accuracy after 50 epochs. About one percentage point above softmax regression. The MLP fits the training set to 99.95% and generalizes a bit worse than that; the gain over the linear model is real but modest. Why is the gap so small, when the MLP is so much more expressive than the linear classifier? Section 7.

6. Logits are log-probabilities, up to a constant

Take the log of the softmax:

$$\log p_i = z_i - \log \sum_j e^{z_j}.$$

So the logits are *not* the log-probabilities. They are the log-probabilities plus a shared term $\log \sum_j e^{z_j}$, the same for every class i . The logits are **unnormalized log-probabilities**.

That constant cannot be recovered from $\mathbf{p}$. Softmax is unchanged if you add any constant c to every logit: e^{z_i+c} has a factor e^c on top and bottom, and it cancels. Training therefore can never pin the constant down. What training *does* pin down is logit **differences**:

$$z_i - z_j = \log p_i - \log p_j = \log \frac{p_i}{p_j},$$

the log-odds between classes. The constant is gauge: free, unobservable.

One sharp way to see that logits are not raw log-probabilities: a probability is at most 1, so a log-probability is at most 0. But nothing stops a network’s logits from being positive. Indeed, if you put a ReLU just before the output, every logit is at least 0. They still describe a perfectly good distribution, because the additive constant absorbs the offset. Logits live on a different scale than log-probabilities; only their differences are shared between them.

7. Inductive biases

The MLP on digits is essentially saturated at 96% test accuracy. Adding more hidden units or more layers does not close the gap to 100%. The model is plenty expressive (it fits the training set to 99.95%), and yet it does not generalize as well as we might hope. The bottleneck is not capacity. It is the **prior** the architecture and the loss commit to.

Inductive bias enters at two distinct points in every supervised model:

1. **The output head:** a choice of link function and matching loss. Identity + MSE assumes Gaussian targets; sigmoid + BCE assumes Bernoulli; softmax + cross-entropy assumes Categorical. The link function is a prior about the data-generating process on the output side.
2. **The architecture:** a choice of how the network computes features. An MLP assumes every input is independent of every other input and learns their relationships from data. A CNN assumes nearby pixels matter together and the same detector should fire anywhere. An RNN assumes the same computation runs at every step of a sequence. A Transformer assumes positions interact pairwise through attention.

Both axes are inductive biases. Both improve sample efficiency when matched to the data and hurt when mismatched. The two are independent: any architecture can pair with any output head, as long as the head’s input dimension matches the architecture’s output.

For the rest of this section we focus on the architecture axis. The output-head axis gets its own dedicated follow-up post.

For the 64 pixels of a digit, the MLP architecture says every feature is independent of every other feature. That is a strong claim, and it happens to be wrong. Pixel (3,4) and pixel (3,5) are neighbors, but the MLP treats them as completely unrelated features and has to relearn their relationship from 3823 examples. Pixel (3,4) and pixel (7,0) are unrelated, but the MLP cannot tell that apart from (3,4) and (3,5) either.

Different architectures commit to different invariances:

- A **CNN** (convolutional network) commits to **locality** and **translation equivariance**: nearby pixels matter together, and a feature detector that fires at one position should fire at any position. This is the right prior for image data.
- An **RNN** commits to **time-translation equivariance**: the same computation runs at every step of a sequence, and the hidden state summarizes the past. The right prior for sequential data.

- A **Transformer** commits to **permutation equivariance** over positions, with **positional encoding** added back to recover a sequence prior. Pairwise interactions are explicit through attention rather than recurrence.
- A **graph neural network** commits to **permutation equivariance** over graph nodes: the answer should not depend on how we ordered the nodes.

These are not freebies. They are bets. A CNN beats an MLP on images *because* its prior matches the data. On tabular data with no spatial structure, a CNN loses to an MLP, because the spatial prior is a lie. The right prior is empirical, not universal.

There is a data-side counterpart. **Data augmentation** expresses the same prior, but from the data instead of the architecture. Random rotations of training images approximate “the answer should not depend on rotation” through the loss: a rotation-equivariant architecture and rotation-augmented data are two routes to the same prior, and they often compose.

A concrete illustration for digits: a single 3×3 convolutional filter applied across the 8×8 image with 16 output channels has roughly 160 weights. The MLP we trained has roughly 2500. The CNN ends up beating the MLP with one-fifteenth the parameters, *because* it has committed to the right prior and so does not have to learn translation invariance from scratch for every position. That is the whole story of inductive bias in one number.

A reasonable question, then, is: how do we choose which prior to commit to? In practice, the data tells you. Images have spatial structure; sequences have temporal structure; graphs have relational structure. The architectures match. When you do not know the structure, an MLP is the safe bet on the architecture side, and identity + MSE is the safe bet on the output side: both commit to nothing and pay for that with data efficiency.

The next several posts walk through the two axes one at a time. **Output heads as inductive bias** covers the link-function axis in depth (Poisson for counts, heteroscedastic Gaussian for uncertainty, bounded outputs for proportions). The architecture axis is split across posts on **convolutional networks** for images, a **fixed-context language model** and **recurrent networks** for sequences, and a **transformer** capstone for attention-based sequence modeling. A closing post on **reinforcement learning** introduces the third learning paradigm beyond supervised. Each is the MLP foundation with a sharper prior baked in at one of the two axes. The math we derived in sections 1 through 6 (forward, backward, chain rule) carries over unchanged. Only the layer types and output heads change.

8. Numerical stability is the gauge freedom, reused

e^z overflows for moderately large z . The fix for softmax: subtract the largest logit first,

$$\text{softmax}(\mathbf{z}) = \text{softmax}\left(\mathbf{z} - \max_i z_i\right).$$

This is *exactly* the additive-constant freedom from section 6. Softmax does not care about a shared shift of the logits, so we spend that freedom to keep the exponentials in a safe range. Stability here is not a new idea bolted on. It is the gauge symmetry put to work.

Cross-entropy is computed as $\text{logsumexp}(\mathbf{z}) - z_y$ rather than $-\log \text{softmax}(\mathbf{z})_y$, which would risk $\log 0$. The sigmoid is evaluated with a sign branch, and binary cross-entropy is computed straight from the logit, both to avoid overflow and $\log 0$.

9. Trust, but verify: numerical gradients

Hand-derived gradients are easy to get subtly wrong. There is a slow but foolproof check: the definition of a derivative itself. For any parameter θ ,

$$\frac{\partial L}{\partial \theta} \approx \frac{L(\theta + \varepsilon) - L(\theta - \varepsilon)}{2\varepsilon}.$$

`gradient_check` computes this central difference for every parameter and compares it to the analytical gradient from `backward`. If they disagree, the analytical gradient is wrong. This is far too slow to train with: one forward pass per parameter. But it is the ground truth that keeps the fast method honest. `test_gradients.py` runs it on all four configurations.

One caveat: `ReLU` has a kink at 0 and no derivative there. A finite difference that straddles the kink will disagree with the analytical sub-gradient. With random weights the probability of a pre-activation landing exactly on the kink is effectively zero, so the check passes in practice. But it is a real reminder that `ReLU` is not differentiable everywhere.

10. Closing note: from per-layer to per-operation

Backpropagation here is per-*layer*: each layer carries a local `backward`. But nothing forced the unit to be a layer. If every scalar *operation* (every `+`, every `×`, every `tanh`) carried its own local backward, the same reverse pass would compute gradients through an arbitrary expression, with no hand-derived layer gradients at all. That is **automatic differentiation**, and it is what every modern framework's autograd engine does. It is the same idea as this library, at a finer grain. Building one is the natural next step.